

Reactive and Responsive Design

Extend CodeBloggs individually with fully functional admin panels, skeleton loaders for reactive UX, and a responsive navigation bar for small screens.

Context

This is Module 10 of the Full-Stack Development Program. You continue in your role as **Junior Developer** at **Genesis Solutions**.

This module continues the CodeBloggs project from Module 9. Students return to individual work. The two admin sections that were left as "under construction" placeholders must now be fully built, and two core UX design patterns must be implemented: **reactive design** (skeleton loaders to improve perceived performance) and **responsive design** (adaptive navigation for different screen sizes).

Client Needs

CodeBloggs has three concerns for this sprint:

1. **Admin functionality**, The User Manager and Content Manager sections must be fully operational. Administrators need to be able to search, edit, delete users, and moderate content (posts and comments) with date filtering and pagination.
2. **Reactive design**, The client is concerned with how the website looks while requests are being processed. They want the site to feel fast and responsive independent of load times. Skeleton screens should appear on admin pages during data fetching and operations.
3. **Responsive design**, The navigation bar on the left takes up too much space on small screens. For screen widths below a defined threshold, the vertical sidebar must be replaced with a horizontal navigation bar.

The client also requires a **written research document** comparing reactive and responsive design.

Global Objectives

Through completion of this business simulation, students will demonstrate skills in the following areas:

- Reactive design: skeleton loaders, perceived performance, and UX during asynchronous operations.
- Responsive design: CSS media queries, adaptive layouts, breakpoint selection and justification.
- Pagination: implementing paginated data views with configurable results per page.
- Admin CRUD operations with cascade delete behavior.

Project Objectives

Extend the existing CodeBloggs application by delivering four features:

1. **User Manager**, Paginated user list with search, edit, and delete functionality (admin only).
2. **Content Manager**, Paginated content list with date range filtering and per-item delete (admin only).
3. **Skeleton Loaders**, Reactive skeleton screens on both admin pages during data loading and delete operations.
4. **Responsive Navbar**, Horizontal navigation bar replacing the vertical sidebar below a justified screen width threshold.

Skeleton loaders must appear immediately on any data fetch or active operation — no blank or broken states visible to the user. Deleting a user cascades to all their posts and comments. The responsive breakpoint threshold must be justified in the README.

Starting Point

No new repository. Students continue individually from the Module 09 CodeBloggs project, transitioning from team to solo work. The same codebase is extended with new admin features and UX patterns.

Expected Deliverables

- Extended CodeBloggs with admin panels (User Manager, Content Manager), skeleton loaders, and responsive navbar.
- The `README.md` that clearly explains the project to someone with no prior knowledge of the setup (must include reactive vs. responsive explanation, breakpoint justification, and setup instructions).
- The global AI specification document (`./ai/ai-spec.md`) and all required feature specification documents (`./ai/features/`).
- The wireframe analysis document covering API design for each new admin screen (backend).
- A Postman collection covering all endpoints, exported as `PostmanCollection.json`.
- The `CONCEPTS.md` listing 3 challenging concepts, with a recorded video demonstrating understanding.
- The LeetCode challenge solutions with a recorded video explaining the reasoning, and solution screenshots saved to `./LeetCode-Challenges/<challenge-name>.png`.
- The Technical Demonstration and Code Overview video of the project.
- An updated résumé incorporating the improvement feedback received submitted to **Asche Binger** on Slack before delivery date.
- Any completed and implemented extra miles.

Requirements

The [Requirement Checklist](#) is the line-by-line source of truth for graded evaluation. If there is any conflict between this document and the checklist, the checklist takes precedence.

The checklist evaluates across seven categories: project setup, technical implementation, AI-native specifications, technical interview, deadline compliance, extra miles, and professional skills.

Constraints

- **Work mode:** Individual, this module is no longer team-based.
- **Parallel development:** Frontend and backend must be developed in parallel, commit history must demonstrate simultaneous progress on both sides, not one completed before the other.
- **Technology:** Continuing with the MERN stack, React Bootstrap, Redux + redux-thunk, react-use-cookie.
- **Color palette:** Must maintain the CodeBloggs palette from Module 9.

- **Branching Model:** Feature branches (`feature/*`) created from `dev`, merged back to `dev`, then `dev` → `main`. No direct commits to `main`.
- **Grading:** Only the `main` branch will be evaluated.
- **Deadline:** The project must be submitted through the platform by **Friday at 11:59 PM** of the assigned deadline week.
- **Submission Summary:** Must NOT be committed to GitHub, it is submitted separately through the platform.

Professional Expectations

This program evaluates both technical and professional skills. Students are expected to:

- Respond to coaches within **24 hours** when contacted.
- Provide at least **2 progress updates per week**.
- Schedule at least **1 project review per week** before Friday.
- Demonstrate autonomy, initiative, professionalism, and attention to detail.

Failure to meet communication and progress requirements may result in a failed module.

References

- The **Platform Slides** provide teaching guidance, concepts, and setup instructions.
- The **Business Document** is the client project brief, it tells the developer what to build, for whom, and to what standard.
- The [Requirement Checklist](#) for precise information of the requirements to implement.